HOMEWORK 3

Elena Pelicella
ele.pelicella@stud.uniroma3.it
604853
https://github.com/elepelix/homework3_elasticsearch

Architettura e implementazione del lavoro
L'obiettivo dell'homework consiste nel progettare e realizzare un motore di indicizzazione e ricerca per gestire un dataset di documenti di testo (.txt). Per condurre una valutazione quanto più significativa, ho utilizzato un dataset pubblico "20 Newsgroups" scaricato da Kaggle in quanto ho impostato il lavoro su un PC nuovo privo di un volume sufficiente di file. Il dataset è articolato in 20 file di testo collocati nella cartella dati/, ognuno dei quali corrisponde a una specifica categoria tematica (ad esempio comp.graphics.txt per la computer grafica, sci.space.txt per scienza e astronomia e comp.sys.ibm.pc.hardware.txt per l'hardware PC) e aggrega al suo interno migliaia di messaggi e discussioni in lingua inglese, per un volume complessivo di 1.365.296 righe non vuote.

Per la realizzazione del progetto ho utilizzato Elasticsearch distribuito ed eseguito in locale all'interno di un container Docker (raggiungibile all'indirizzo http://localhost:9200), scelta che ha reso l'avvio del servizio immediato e senza bisogno di installazioni sul sistema operativo.

Ho sviluppato l'intera architettura in Python, utilizzato sia per realizzare l'interfaccia di ricerca interattiva destinata all'utente, sia come client HTTP (tramite la libreria ufficiale elasticsearch) per comunicare via API REST con il motore di ricerca. Per garantire modularità e pulizia del codice, il progetto è stato suddiviso in due file Python distinti all'interno della cartella scripts/:
- indexer.py: dedicato esclusivamente alla configurazione degli indici, alla definizione dei mapping e al caricamento dei dati.
- searcher.py: dedicato all'interfaccia di ricerca CLI (Command Line Interface), responsabile del parsing delle query dell'utente e dell'invio delle richieste ad Elasticsearch.

In linea con la richiesta di gestire separatamente i nomi dei file e il loro contenuto, ho definito due indici distinti:
- index_nomi: relativo all'indicizzazione dei file, associa a ogni documento l'attributo filename.
- index_testi: relativo al contenuto testuale del file; ho scelto di indicizzare ogni singola riga non vuota come documento separato (associando filename, contenuto e numero di linea), così da poter restituire nei risultati di ricerca anche la riga esatta in cui compare il testo.

Analyzer
Per ottimizzare la ricerca in base alla tipologia e alla natura del dato, ho definito i seguenti analyzer nella configurazione dello schema degli indici:

- Per l’indice index_nomi
  - Analyzer utilizzato: keyword
  - Motivazione della scelta: ho scelto di privilegiare la ricerca per nome esatto. Keyword conserva infatti il nome del file nella sua interezza, evitando la frammentazione del testo sui punti o sui caratteri speciali e consentendo il match esatto sulla stringa originale del file.

- Per l’indice index_testi
  - Analyzer utilizzato: english
  - Motivazione della scelta: il dataset scelto è composto da file di testo in lingua inglese e l’analyzer english garantisce una pipeline completa per il retrieval full-text. In particolare divide il testo in parole (tokenizzazione) e lo converte in minuscolo (lowercasing) rendendo le ricerche case-insensitive; elimina le parole molto comuni e prive di significato (stop-words removal, ad esempio “is”, “the”…); riduce i termini alle loro radici morfologiche (stemming) così da garantire che la ricerca per un determinato termine individui anche le sue varianti al plurale o più in generale le sue forme flesse, incrementando il recall del sistema.

Metriche di indicizzazione e prestazioni
Per ottimizzare il processo e velocizzare i tempi di caricamento, invece di optare per la chiamata alla singola API es.index(), che avrebbe generato oltre un milione di richieste HTTP individuali introducendo inevitabilmente un overhead di rete, ho utilizzato la funzione bulk() del modulo elasticsearch.helpers. Questo metodo raccoglie le righe in memoria e le invia ad Elasticsearch a blocchi in poche richieste complessive, riducendo drasticamente il tempo totale di caricamento.
Metriche rilevate durante l’esecuzione:
- Cartella sorgente: dati/
- Numero di file indicizzati: 20
- Righe non vuote totali elaborate e indicizzate: 1.365.296
- Tempo totale di indicizzazione: 22.50 secondi
- Tempi di risposta delle query: compresi tra 2 ms e 15 ms (verificati tramite il metadato took restituito da Elasticsearch).

Verifica e risultati delle query
Per verificare la correttezza dello schema degli indici, il comportamento degli analyzer e la gestione delle interrogazioni, ho eseguito 13 query rappresentative. Di seguito sono riportati i risultati reali ottenuti dall’esecuzione e la relativa analisi (sottolineo che, per la natura del dataset originale 20 Newsgroups, alcune discussioni o messaggi citati risultano duplicati a righe diverse all'interno dello stesso file):

1. Query: contenuto: "anti-aliasing"
   - Indice interrogato: index_testi
   - Risultati trovati: 45 (tempo di risposta: 15 ms)
   - Analisi: Trattandosi di una stringa tra virgolette, lo script ha eseguito una match_phrase query. L'analyzer english ha gestito il trattino dividendo i termini, mentre la match_phrase ha garantito la ricerca della sequenza contigua dei termini stemmati.

2. Query: contenuto: "scientific visualization"
   - Indice interrogato: index_testi
   - Risultati trovati: 81 (tempo di risposta: 6 ms)
   - Analisi: La query match_phrase ha intercettato correttamente le righe in cui le parole "scientific" e "visualization" compaiono consecutivamente nel testo.

3. Query: nome: comp.os.ms-windows.misc.txt
   - Indice interrogato: index_nomi
   - Risultati trovati: 1 (tempo di risposta: 2 ms)
   - Analisi: L'analyzer keyword ha preservato l'integrità del nome del file senza spezzarlo sui punti e sui trattini, garantendo un match esatto e univoco.

4. Query: contenuto: raytracing
   - Indice interrogato: index_testi
   - Risultati trovati: 183 (tempo di risposta: 4 ms)
   - Analisi: Esecuzione di una match query standard su termine singolo. Grazie allo stemming dell'analyzer english, la ricerca ha individuato sia "raytracing" sia varianti come "Raytracers", ricondotte alla stessa radice.

5. Query: contenuto: "texture mapping"
   - Indice interrogato: index_testi
   - Risultati trovati: 51 (tempo di risposta: 5 ms)
   - Analisi: Query di tipo match_phrase. Poiché l'analyzer english applica lo stemming anche alle phrase query, "mapping" viene ridotto alla radice "map": di conseguenza la sequenza esatta viene verificata sui termini stemmati, intercettando sia "texture mapping" sia espressioni come "texture map" (aumentando il recall ma con un possibile lieve calo di precision).

6. Query: contenuto: "Delaunay triangulation"
   - Indice interrogato: index_testi
   - Risultati trovati: 33 (tempo di risposta: 4 ms)
   - Analisi: Evidenzia l'efficacia del lowercasing dell'analyzer english: la query inviata con la "D" maiuscola ha intercettato sia "Delaunay Triangulation" che "Delaunay triangulation".

7. Query: contenuto: "packet driver"
   - Indice interrogato: index_testi
   - Risultati trovati: 98 (tempo di risposta: 4 ms)
   - Analisi: L'impiego della match_phrase query garantisce che i due termini vengano individuati esclusivamente come sequenza contigua (verificata sulle radici stemmate, intercettando quindi sia "packet driver" che "Packet Drivers"). I 98 riscontri evidenziano la capacità del sistema di recuperare un concetto tecnico specifico presente in documenti appartenenti a macro-aree tematiche differenti, come l'hardware PC (comp.sys.ibm.pc.hardware.txt) e i sistemi di interfaccia grafica (comp.windows.x.txt).

8. Query: contenuto: "virtual memory"

- Indice interrogato: index_testi
- Risultati trovati: 69 (tempo di risposta: 4 ms)
- Analisi: La ricerca per frase esatta presente in contesti eterogenei conferma la trasversalità dell'indice.

9. Query: contenuto: DoubleSpace
- Indice interrogato: index_testi
- Risultati trovati: 52 (tempo di risposta: 3 ms)
- Analisi: Dimostra l'azione combinata di lowercasing e stemming dell'analyzer english. La ricerca di "DoubleSpace" ha individuato sia "doublespace" sia le forme flesse "DoubleSpaced" e "doublespacing".

10. Query: nome: comp.graphics.txt
- Indice interrogato: index_nomi
- Risultati trovati: 1 (tempo di risposta: 2 ms)
- Analisi: Match esatto ed atomico sul nome del file tramite l'analyzer keyword.

11. Query: contenuto: space
- Indice interrogato: index_testi
- Risultati trovati: 5019 (tempo di risposta: 5 ms)
- Analisi: Termine ad alta frequenza che mostra la capacità del sistema di gestire grandi volumi di corrispondenze su più categorie.

12. Query: contenuto: system
- Indice interrogato: index_testi
- Risultati trovati: 10000 (tempo di risposta: 3 ms)
- Analisi: Termine ad altissima frequenza. Per impostazione predefinita (parametro `track_total_hits`), Elasticsearch interrompe il conteggio esatto dei risultati a 10.000 per ottimizzare le prestazioni: il valore riportato rappresenta quindi un valore inferiore al totale reale.

13. Query: contenuto: graphics
- Indice interrogato: index_testi
- Risultati trovati: 3726 (tempo di risposta: 4 ms)
- Analisi: Dimostra l'efficacia dello stemming (riconduce "graphics" e "graphic" alla medesima radice) recuperando numerose varianti e occorrenze nel testo e incrementando il recall generale della ricerca.

Considerazioni, limiti e sviluppi
Il sistema ha dimostrato ottime prestazioni sia in fase di ingestione (grazie all'utilizzo della funzione bulk() che ha permesso di caricare oltre 1,36 milioni di righe in circa 22 secondi) sia in fase di interrogazione, garantendo risposte nell'ordine dei pochi millisecondi.

Tuttavia, l'attuale implementazione presenta alcuni aspetti strutturali migliorabili in futuro:

- Granularità per singola riga: indicizzare ogni riga come documento separato consente di indicare il numero di linea nei risultati, ma impedisce alle match_phrase query di individuare frasi spezzate su due righe consecutive.

- Ricerca esatta sui nomi dei file: l'analyzer keyword su index_nomi permette esclusivamente match esatti e case-sensitive (ad esempio, cercare nome: comp.graphics senza estensione non produce risultati). Come sviluppo futuro si potrebbe adottare un analyzer custom con tokenizer su ".", "-" e "_" e filtro lowercase per abilitare ricerche parziali sui nomi.

- Aggiornamento non incrementale: ad ogni esecuzione dello script indexer.py, gli indici esistenti vengono eliminati e ricreati da zero. In uno scenario reale sarebbe opportuno implementare un meccanismo di indicizzazione incrementale per aggiungere o aggiornare solo i file nuovi o modificati.

- Bilanciamento tra recall e precision: l'uso dell'analyzer english incrementa sensibilmente il recall (intercettando forme flesse e plurali), ma in alcuni casi può ridurre alla stessa radice morfologica parole con significati differenti determinando dunque cali di precision.

- Gestione delle stopword: query composte esclusivamente da parole molto comuni in lingua inglese come "the", "is" o "and" non producono alcun risultato in quanto termini rimossi sia in fase di indicizzazione che di analisi della query.

- Ricerche avanzate: attualmente l'interfaccia CLI supporta query singole (match e match_phrase); un possibile sviluppo futuro potrebbe consistere nell'integrare query booleane complesse (con operatori AND, OR, NOT) e filtri combinati simultaneamente su nome del file e contenuto.